Language Variants

- Document number: P0095R2
- Date: 2018-10-07

- Reply-to: David Sankel <dsankel@bloomberg.net>, Dan Sarginson<dsarginson@bloomberg.net>, Sergei Murzin<smurzin@bloomberg.net>

- Audience: Evolution

Abstract

Language-based variants extend and enhance the sum type capabilities offered by C++. They do not replace `std::variant`, which still has viable use cases, but they do address a number of the drawbacks of a standard library approach. This paper proposes a syntax that extends C++ to make variants a language-level feature.

History

P0095R2. Split the original paper into individual proposals, keeping this paper only for proposed language variant syntax. Pattern matching for built-in types and opt-in syntax for pattern matching were split into separate papers.

P0095R1. Merged in blog post developments. Added `nullptr` patterns, `@` patterns, and pattern guards. A mechanism for dealing with assignment was also added. Wording as it relates to patterns was added. Made expression and statement `inspects` use a single keyword.

C++ Language Support for Pattern Matching and Variants blog post. Sketched out several ideas on how a more extensive pattern matching feature would look. Discussed an extension mechanism which would allow any type to act tuple-like or variant-like. `lvariant` is used instead of `enum union` based on feedback in Kona.

Kona 2015 Meeting. There was discussion on whether or not a partial pattern-matching solution would be sufficient for incorporation of a language-based variant. While exploration of a partial solution had consensus at 5-12-8-2-0, exploration of a full solution had a strong consensus at 16-6-5-1-0. The question was also asked whether or not we want a language-based variant and the result was 2-19-6-0-1.

P0095R0. The initial version of this paper presented in Kona. It motivated the need for a language-based variant and sketched a basic design for such a feature with the minimal pattern matching required.

Introduction

Standard library variants have provided type safety and expressiveness to the sum type support in C++. This is a good result that has enabled important functional idioms to be used, but as a tool `std::variant` is made less applicable by the limitations placed upon it by the language.

In addition the authors feel that standard library variants are a complicated feature to explain to notice programmers, and fraught with pitfalls and potential bugs.

This paper presents a design for language-level variants that addresses the shortcomings of a pure standard library variant, in a syntax that the authors feel will be elegant for creating sum type solutions and intuitive for C++ programmers at all levels of experience.

The following snippet illustrates our proposed syntax.

```
// This lvariant implements a value representing the various commands
// available in a hypothetical shooter game.
lvariant command {
  std::size_t set_score; // Set the score to the specified value
  std::monostate fire_missile; // Fire a missile
  unsigned fire_laser; // Fire a laser with the specified intensity
  double rotate; // Rotate the ship by the specified degrees.
};

// Output a human readable string corresponding to the specified 'cmd' command
// to the specified 'stream'.
std::ostream& operator<<( std::ostream& stream, const command cmd ) {
  return inspect( cmd ) {
    set_score value =>
      stream << "Set the score to " << value << ".\n"
```

```
    fire_missile m =>
      stream << "Fire a missile.\n"
    fire_laser intensity:
      stream << "Fire a laser with " << intensity << " intensity.\n"
    rotate degrees =>
      stream << "Rotate by " << degrees << " degrees.\n"
  };
}

// Create a new command 'cmd' that sets the score to '10'.
command cmd = command::set_score( 10 );
```

Motivation

The current library-based variants solve an important need, but they are too complicated for novice users. We describe difficult corner cases, the pitfalls of using types as tags, and the difficulty of writing portable code using a library based variant. All of these problems suggest the necessity of a language-based variant feature in C++.

The struct/tuple and lvariant/variant connection

Basic `struct` types that have independently varying member variables[1] have a close relationship to the `std::tuple` class. Consider the following two types:

```
// point type as a struct
struct point {
  double x;
  double y;
  double z;
};

// point type as a tuple
using point = std::tuple< double, double, double >;
```

It is clear that both `point` types above can represent a 3D mathematical point. The difference between these two types is, essentially, the tag which is used to discriminate between the three elements. In the `struct` case, an identifier is used (x, y, and z), and in the `std::tuple` case, an integer index is used (0, 1, and 2).

Although these two `point` implementations are more-or-less interchangeable, it is not always preferable to use a `struct` instead of a `std::tuple` nor vise-versa. In particular, we have the following general recommendations:

1. If the type needs to be created on the fly, as in generic code, a `std::tuple` must be used.
2. If an integer index isn't a clear enough identifier, a `struct` should be used.
3. Arguably, if inner types aren't essentially connected or if the structure is used only as the result of a function and is immediately used, a `std::tuple` is preferable.
4. In general, prefer to use a `struct` for improved code clarity.

Some may argue that through use of `std::get`, which allows one to fetch a member of a tuple by type, one can achieve all the benefits of a `struct` by using a tuple instead. To take advantage of this feature, one needs to ensure that each inner type has its own distinct type. This can be accomplished through use of a wrapper. For example:

```
struct x { double value; };
struct y { double value; };
struct z { double value; };

using point = std::tuple< x, y, z >;
```

Now one could use `std::get<x>` to fetch the 'x' value of the tuple, `std::get<y>` for 'y' and so on.

Should we use this approach everywhere and deprecate the use of `struct` in any context? In the authors' opinion we should not. The use of wrapper types is much more complicated to both read and understand than a plain `struct`. For example, the wrapper types that were introduced, such as the 'x' type, make little sense outside of their corresponding tuples, yet they are peers to it in scope. Also, the heavy syntax makes it difficult to understand exactly what is intended by this code.

What does all this have to do with lvariants? The lvariant is to `std::variant` as `struct` is to `std::tuple`. A variant type that represents a distance in an x direction, a y direction, *or* a z direction (mathematically called a "copoint") has a similar look and feel to the `std::tuple` version of `point`.

```
struct x { double value; };
struct y { double value; };
struct z { double value; };
```

```
using copoint = std::variant< x, y, z >;
```

This copoint implementation has the same drawbacks that the `std::tuple` implementation of points has. An lvariant version of `copoint`, on the other hand, is easier to grok and doesn't require special tag types at all.

```
lvariant copoint {
  double x;
  double y;
  double z;
};
```

SFINE in basic usage

Some variation of the following example is common when illustrating a `std::variant` type:

```
void f( std::variant< double, std::string> v ) {
  if( std::holds_alternative< double >( v ) {
    std::cout << "Got a double " << std::get< double >( v ) << std::endl;
  }
  else {
    std::cout << "Got a string " << std::get< std::string >( v ) << std::endl;
  }
};
```

This illustrates how quickly variants can be disassembled when they are simple, but it is hardly representative of how complex variant types are used. The primary problem in the above snippet is that there are no compile-time guarantees that ensure all of the `n` alternatives are covered. For the more general scenario, a `visit` function is provided.[2]

```
struct f_visitor {
  void operator()( const double d ) {
    std::cout << "Got a double " << d << std::endl;
  }
  void operator()( const std::string & s ) {
    std::cout << "Got a string " << s << std::endl;
  }
};

void f( std::variant< double, std::string > v ) {
  std::visit( f_visitor(), v );
};
```

Aside from the unsightly verbosity of the above code, the mechanism by which this works makes the visitor's `operator()` rules work by SFINE, which is a significant developer complication. Using a template parameter as part of a catch-all clause is going to necessarily produce strange error messages.

```
struct f_visitor {
  template< typename T >
  void operator()( const T & t ) {
                                       // oops
    std::cout << "I got something " << t.size() << std::endl;
  }
};

void f( std::variant< double, std::string > v ) {
  // Unhelpful error message awaits. Erroneous line won't be pointed out.
  std::visit( f_visitor(), v );
};
```

While the utility of type selection and SFINE for visitors is quite clear for advanced C++ developers, it presents significant hurdles for the beginning or even intermediate developer. This is especially true when it is considered that the `visit` function is the only way to guarantee a compilation error when all cases are not considered.

Duplicated types: switching on the numeric index

Using types as accessors with a `std::variant` works for many use cases, but not all. If there is a repeated type the only options are to either use wrapper types or to work with the real underlying discriminator, an integer index. To illustrate the problems with using the index, consider the following implementation of copoint:

```
using copoint = std::variant< double, double, double >;
```

Use of both `std::get<double>` and the standard `std::visit` are impossible due to the repeated `double` type in the variant. Using the numeric index to work around the issue brings its own problems, however. Consider the following visitor:

```
struct visit_f {
```

```
  void operator()( std::integral_constant<std::size_t, 0>, double d ) {
    std::cout << d << " in x" << std::endl;
  };
  void operator()( std::integral_constant<std::size_t, 1>, double d ) {
    std::cout << d << " in y" << std::endl;
  };
  void operator()( std::integral_constant<std::size_t, 2>, double d ) {
    std::cout << d << " in z" << std::endl;
  };
};
```

Here we introduce yet another advanced C++ feature, compile-time integrals. In the opinion of the author, this is unfriendly to novices. The problem of duplicated types can be even more insidious, however...

Portability problems

Consider the following code:

```
using json_integral = std::variant< int, unsigned, std::size_t, std::ptr_diff_t >;
```

On most platforms, this code will compile and run without a problem. However, if `std::size_t` happens to be `typedef`'d to be the same type as `unsigned` on a particular platform, a compilation error will ensue. The only two options for fixing the error are to fall back to using the index or to make custom wrapper types, and this is assuming one *can* edit the library type.

Also notable is that working with third party libraries that are free to change their underlying types creates abstraction leaks when used with a library-based variant.

```
// Is this code future proof? Not likely. Looks like a foot-gun to me.
using database_handle = std::variant< ORACLE_HANDLE, BERKELEY_HANDLE >;
```

Because lvariants require identifiers as tags, they aren't susceptible to this problem:

```
lvariant database_handle {
  ORACLE_HANDLE oracle;
  BERKELEY_HANDLE berkeley;
};
```

Language Based Variant `lvariant`

The definition of an lvariant has the same syntax as a `union`, but with an `lvariant` keyword as in the following example:

```
// This lvariant implements a value representing the various commands
// available in a hypothetical shooter game.
lvariant command {
  std::size_t set_score; // Set the score to the specified value
  std::monostate fire_missile; // Fire a missile
  unsigned fire_laser; // Fire a laser with the specified intensity
  double rotate; // Rotate the ship by the specified degrees.
};
```

Each member declaration consists of a type followed by its corresponding identifier.

Construction and Assignment

An lvariant has a default constructor if its first field also has a default constructor. A default constructed lvariant is set to the first fields's default constructed value.

Assignment at construction can be used to set the lvariant to a particular value. The lvariant is used as a namespace when specifying specific alternatives.

```
command cmd = command::set_score( 10 );
```

lvariant instances can also be assigned in the course of a program's execution.

```
cmd = command::fire_missile( );
```

Inspection

Extracting values from an lvariant is acomplished with a new `inspect` keyword. While pattern matching is covered in an accompanying paper P1308, we provide some basic examples below for exposition purposes.

```
inspect( cmd ) {
```

```
  set_score value =>
    stream << "Set the score to " << value << ".\n";
  fire_missile m =>
    stream << "Fire a missile.\n";
  fire_laser intensity =>
    stream << "Fire a laser with " << intensity << " intensity.\n";
  rotate degrees =>
    stream << "Rotate by " << degrees << " degrees.\n";
}
```

Assignment

As with library-based variants, the behavior of assignment when an exception is thrown is of considerable concern. We propose the following for lvariants:

- If any of the alternatives is not friendly (ie. has a possibly throwing move constructor or a possibly throwing move assignment operator), there will not be a default assignment operator for the lvariant.
- Users will have the ability to implement their own assignment operator to their liking.

This provides a safe default and supports users of differing philosophies.

The “I’m broken. You deal with it.” philosophy allows the `lvariant` to get into a state where the only valid operations are assignment and destruction. This is accomplished by overriding the assignment operator and allowing the ‘std::valueless_by_exception’ exception to pass through to callers.

```
lvariant Foo {
  PossiblyThrowingMoveAssignmentType field1;
  std::string field2;

  // Possibly throw a 'std::valueless_by_exception' exception which makes this
  // object only assignable and destructable.
  Foo& operator=(const Foo& rhs);
  Foo& operator=(const Foo&& rhs); // implementation skipped
};

Foo& Foo::operator=(const Foo& rhs)
{
  // This can possibly throw a 'std::valueless_by_exception' exception.
  lvariant(*this) = rhs;
}
```

The “exception are for the weak” philosophy essentially terminates the program if there’s an exception on assignment. This is accomplished by marking the assignment operator `noexcept`.

```
lvariant Foo {
  PossiblyThrowingMoveAssignmentType field1;
  std::string field2;

  Foo& operator=(const Foo& rhs) noexcept;
  Foo& operator=(const Foo&& rhs) noexcept; // implementation skipped
};

Foo& operator=(const Foo& rhs) noexcept
{
  lvariant(*this) = rhs;
}
```

The “embrace emptiness” philosophy switches to a special empty state if there’s an exception on assignment. This is accomplished by handling the `std::valueless_by_exception` exception within the assignment operator.

```
lvariant Foo {
  PossiblyThrowingMoveAssignmentType field1;
  std::string field2;
  std::monostate empty;

  Foo& operator=(const Foo& rhs);
  Foo& operator=(const Foo&& rhs); // implementation skipped
};

Foo& operator=(const Foo& rhs)
{
  try {
    lvariant(*this) = rhs;
  }
  catch(std::valueless_by_exception&) {
```

```
    lvariant(*this) = Foo::empty();
  }
}
```

Pattern matching `lvariants`

Pattern matching is the easiest way to work with `lvariants`. Consider the following binary tree with `int` leaves.

```
lvariant tree {
  int leaf;
  std::pair< std::unique_ptr<tree>, std::unique_ptr<tree> > branch;
}
```

Say we need to write a function which returns the sum of a `tree` object's leaf values. Variant patterns are just what we need. A pattern which matches an alternative consists of the alternative's name followed by a pattern for its associated value.

```
int sum_of_leaves( const tree & t ) {
  return inspect( t ) {
           leaf i => i
           branch b => sum_of_leaves(*b.first) + sum_of_leaves(*b.second)
         };
}
```

Assuming we can pattern match on the `std::pair` type, which is discussed in the companion paper, this could be rewritten as follows.

```
int sum_of_leaves( const tree & t ) {
  return inspect( t ) {
           leaf i => i
           branch [left, right] => sum_of_leaves(*left) + sum_of_leaves(*right)
         };
}
```

Conclusion

We conclude that types-as-tags are for astronauts, but variants are for everyone. None of the library implementations thus far proposed are easy enough to be used by beginners; a language feature is necessary. In the authors' opinion a library-based variant should complement a language-based variant, but not replace it. And with language-based variants comes pattern matching, another highly desirable feature in the language.

Acknowledgements

Thanks to Vicente Botet Escribá, John Skaller, Dave Abrahams, Bjarne Stroustrup, Bengt Gustafsson, and the C++ committee as a whole for productive design discussions. Also, Yuriy Solodkyy, Gabriel Dos Reis, and Bjarne Stroustrup's prior research into generalized pattern matching as a C++ library has been very helpful.

References

- V. Botet Escribá. Product types access. P0327R0. WG21
- V. Botet Escribá. Structured binding: customization points issues. P0326R0. WG21
- D. Sankel. C++ Langauge Support for Pattern Matching and Variants. davidsankel.com.
- Y. Solodkyy, G. Dos Reis, B. Stroustrup. Open Pattern Matching for C++. GPCE 2013.
- H. Sutter, B. Stroustrup, G. Dos Reis. Structured bindings. P0144R2. WG21.

Appendix 1: Before/After Comparisons

Figure 1. Declaration of a command data structure.

before	after

`struct set_score {`
`  std::size_t value;`
`};`
 	
`struct fire_missile {};`
 	
`struct fire_laser {`
`  unsigned intensity;`
`};`
	`lvariant command {`
	`  std::size_t set_score;`
	`  std::monostate fire_missile;`
	`  unsigned fire_laser;`
	`  double rotate;`
	`};`

```
struct rotate {
  double amount;
};

struct command {
  std::variant<
    set_score,
    fire_missile,
    fire_laser,
    rotate > value;
};
```

Figure 2: Implementation of an output operator.

before

```
namespace {
struct Output {
  std::ostream& operator()(std::ostream& stream, const
  set_score& ss) const {
    return stream << "Set the score to " << ss.value <<
  ".\n";
  }
  std::ostream& operator()(std::ostream& stream, const
  fire_missile&) const {
    return stream << "Fire a missile.\n";
  }
  std::ostream& operator()(std::ostream& stream, const
  fire_laser& fl) const {
    return stream << "Fire a laser with " << fl.intensity <<
  " intensity.\n";
  }
  std::ostream& operator()(std::ostream& stream, const
  rotate& r) const {
    return stream << "Rotate by " << r.degrees << "
  degrees.\n"
  }
};
}

std::ostream& operator<<(std::ostream& stream, const command&
  cmd) {
  return std::visit(std::bind(Output(), std::ref(stream),
  std::placeholders::_1),
                    cmd.value);
}
```

after

```
std::ostream& operator<<(std::ostream& stream,
  const command& cmd) {
  return inspect(cmd) {
    set_score value =>
      stream << "Set the score to " << value <<
  ".\n"
    fire_missile _ =>
      stream << "Fire a missile.\n"
    fire_laser intensity =>
      stream << "Fire a laser with " << intensity
  << " intensity.\n"
    rotate degrees =>
      stream << "Rotate by " << degrees << "
  degrees.\n"
  }
}
```

Figure 3: Expression Datatype.

before

```
struct expression;

struct sum_expression {
  std::unique_ptr<expression> left_hand_side;
  std::unique_ptr<expression> right_hand_side;
};

struct expression {
  std::variant<sum_expression, int, std::string> value;
};

expression simplify(const expression & exp) {
  if(sum_expression const * const sum =
  std::get_if<sum_expression>(&exp)) {
    if( int const * const lhsInt = std::get_if<int>( sum-
  >left_hand_side.get() )
      && *lhsInt == 0 ) {
      return simplify(*sum->right_hand_side);
    }
    else if( int const * const rhsInt = std::get_if<int>( sum-
  >right_hand_side.get() )
             && *rhsInt == 0 ) {
      return simplify(*sum->left_hand_side);
    } else {
      return {sum_expression{
```

after

```
lvariant expression;

struct sum_expression {
  std::unique_ptr<expression> left_hand_side;
  std::unique_ptr<expression> right_hand_side;
};

lvariant expression {
  sum_expression sum;
  int literal;
  std::string var;
};

expression simplify(const expression & exp) {
  return inspect(exp) {
           sum [*(literal 0),         *rhs] =>
  simplify(rhs)
           sum [*lhs        , *(literal 0)] =>
  simplify(lhs)
           sum [*lhs        ,         *rhs]
             => expression::sum{
                  std::make_unique<expression>
  (simplify(lhs)),
                  std::make_unique<expression>
  (simplify(rhs))};
           _ => exp
```

```
        std::make_unique<expression>(simplify(*sum-
  >left_hand_side)),
        std::make_unique<expression>(simplify(*sum-
  >right_hand_side))}}
    }
  }
  return exp;
}

void simplify2(expression & exp) {
  if(sum_expression * const sum = std::get_if<sum_expression>
  (&exp)) {
    if( int * const lhsInt = std::get_if<int>( sum-
  >left_hand_side.get() )
        && *lhsInt == 0 ) {
      expression tmp(std::move(*sum->right_hand_side));
      exp = std::move(tmp);
      simplify(exp);
    }
    else if( int * const rhsInt = std::get_if<int>( sum-
  >right_hand_side.get() )
             && *rhsInt == 0 ) {
      expression tmp(std::move(*sum->left_hand_side));
      exp = std::move(tmp);
      simplify(exp);
    } else {
      simplify(*sum->left_hand_side);
      simplify(*sum->right_hand_side);
    }
  }
  return exp;
}
```

```
        };
}

void simplify2(expression & exp) {
  inspect(exp) {
    sum [*(literal 0),          *rhs] => {
      expression tmp(std::move(rhs));
      exp = std::move(tmp);
      simplify2(exp);
    }
    sum [*lhs         , *(literal 0)] => {
      expression tmp(std::move(lhs));
      exp = std::move(tmp);
      simplify2(exp);
    }
    sum [*lhs         ,          *rhs] => {
      simplify2(lhs);
      simplify2(rhs);
    }
    _ => ;
  };
}
```

---

1. See The C++ Core Guidelines rule C.2.↩

2. Compare that code to the same for an lvariant:

```
lvariant double_or_string {
  double with_double;
  std::string with_string;
};

void f( double_or_string v ) {
  switch( v ) {
    case with_double d:
      std::cout << "Got a double " << d << std::endl;
    case with_string s:
      std::cout << "Got a string " << s << std::endl;
  }
}
```

↩